



FOOD STORE

Sistema de Gestión de Pedidos de Comida

Especificación Técnica del Sistema

Versión 5.0 · Spec-Driven Development (SDD) · Feature-First

Campo	Valor
Materia	
Carrera	
Modalidad	Trabajo Práctico Integrador (TPI)
Stack	React + TypeScript + FastAPI + PostgreSQL
Metodología	Spec-Driven Development (SDD)
Versión doc.	5.0 — ERD v5 + Arquitectura SDD + Diagramas + Feature-First

1. Visión General del Sistema

Food Store es una aplicación web full-stack para la gestión integral de un negocio de comidas. Permite a los clientes explorar el catálogo, agregar productos al carrito, realizar pedidos con pago integrado vía MercadoPago y hacer seguimiento en tiempo real del estado de su pedido. Los administradores gestionan el catálogo, el stock, los pedidos y los usuarios desde un panel centralizado.

1.1 Objetivos del Sistema

#	Actor	Objetivo principal
OBJ-01	Cliente	Navegar el catálogo, gestionar carrito, pagar con MercadoPago y rastrear pedidos con trazabilidad completa
OBJ-02	Administrador	Gestionar categorías, productos, stock y ciclo de vida de pedidos desde el panel
OBJ-03	Gestor de Stock	Controlar disponibilidad y cantidad de stock de productos
OBJ-04	Gestor de Pedidos	Avanzar el estado de los pedidos según la máquina de estados definida
OBJ-05	Sistema	Garantizar trazabilidad completa de transiciones de estado mediante audit trail append-only
OBJ-06	Sistema	Procesar y registrar pagos a través de la pasarela MercadoPago de forma atómica

1.2 Alcance v5.0

- Autenticación y autorización con JWT y RBAC (4 roles) + invalidación de refresh token en base de datos
- Catálogo de productos con categorías jerárquicas e ingredientes con campo es_alergeno
- Carrito de compras con persistencia mediante Zustand + localStorage
- Gestión de pedidos con máquina de estados de 6 estados y audit trail append-only
- Pasarela de pagos MercadoPago Checkout API: tarjeta de crédito/débito, Rapipago, Pago Fácil
- Notificaciones webhook IPN de MercadoPago para confirmación automática de pagos
- Módulo DireccionEntrega: CRUD completo con dirección principal por usuario
- Panel de administración: dashboard con recharts, CRUD de entidades, gestión de pedidos y stock
- Rate limiting con slowapi: máximo 5 intentos fallidos por IP en 15 minutos en el login
- CORS configurado correctamente con CORSMiddleware para la separación frontend/backend
- Seed data obligatorio: roles, estados de pedido, formas de pago y usuario administrador
- API REST documentada con FastAPI/OpenAPI — accesible en /docs y /redoc

1.3 Stack Tecnológico

Capa	Tecnología	Versión	Rol en el sistema
Frontend	React + TypeScript	18.x + 5.x	UI, enrutamiento, componentes
Frontend	Vite	5.x	Build tool y dev server
Frontend	Tailwind CSS	3.x	Estilos utility-first
Frontend	TanStack Query	5.x	Fetching, caché y sincronización de datos del servidor
Frontend	TanStack Form	0.x	Gestión de formularios con validación
Frontend	Zustand	4.x	Estado global del cliente (carrito, sesión, pagos, UI)
Frontend	Axios	1.x	Cliente HTTP con interceptors JWT
Frontend	recharts	2.x	Gráficos del dashboard de administración

Capa	Tecnología	Versión	Rol en el sistema
Frontend	@mercadopago/sdk-react	—	SDK oficial MercadoPago para tokenización PCI-compliant
Backend	FastAPI	0.111+	Framework REST + generación automática OpenAPI
Backend	SQLModel	0.0.19+	ORM + schemas Pydantic integrados
Backend	PostgreSQL	15+	Base de datos relacional
Backend	Alembic	1.13+	Migraciones versionadas de base de datos
Backend	Passlib (bcrypt)	—	Hashing de contraseñas (cost factor ≥ 12)
Backend	mercadopago	2.3.0+	SDK oficial MercadoPago Python
Backend	slowapi	0.1.9+	Rate limiting por IP en endpoints críticos

2. Arquitectura del Sistema

2.1 Capas del Backend — Flujo de Dependencias

El backend aplica una arquitectura de capas con módulos por feature. El patrón Unit of Work (UoW) se ubica entre la capa de servicio y los repositorios, garantizando atomicidad transaccional. El flujo de dependencias es unidireccional y no puede invertirse.

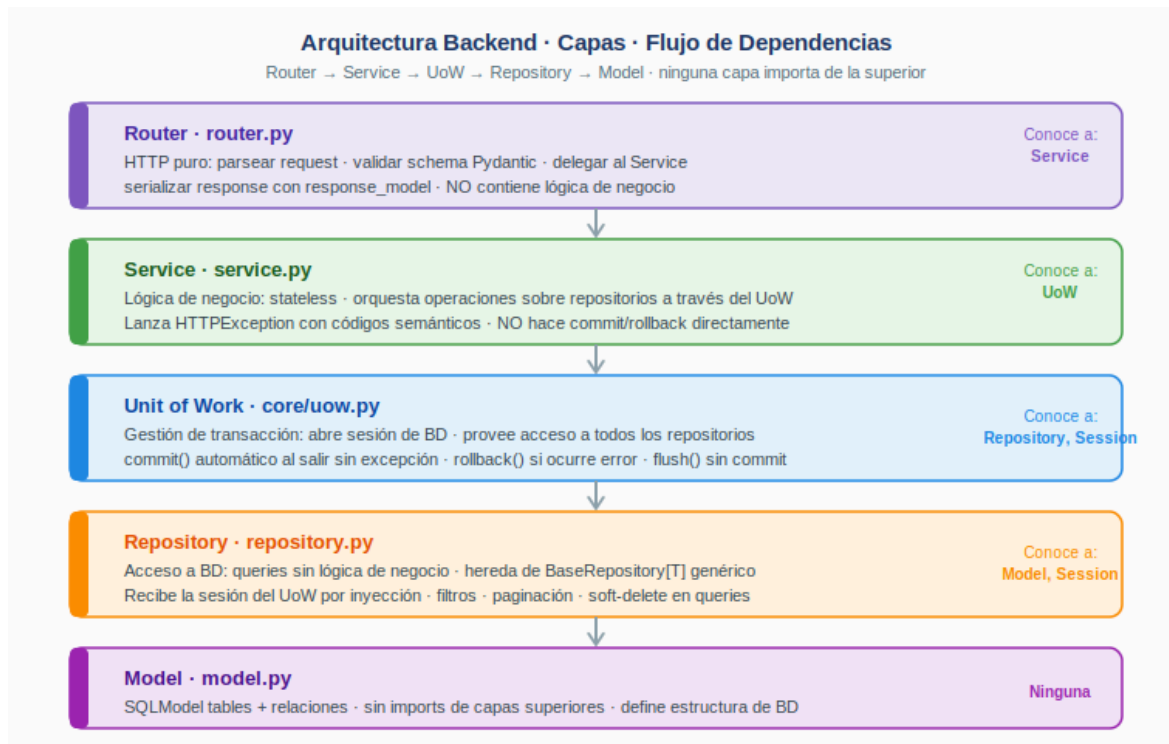


Figura 1 — Capas del backend y flujo de dependencias unidireccional

Regla de oro — flujo de imports:

Router → Service → UoW → Repository → Model

Ninguna capa puede importar de la capa superior.

Un Model nunca importa de un Service. Un Repository nunca importa de un Router.

Capa	Archivo de referencia	Responsabilidad	Conoce a
Router	router.py	HTTP puro: parsear request, validar schema Pydantic, delegar al Service, serializar response con response_model. No contiene lógica de negocio.	Service
Service	service.py	Lógica de negocio: stateless, orquesta operaciones sobre los repositorios a través del UoW. Lanza HTTPException. No hace commit/rollback directamente.	UoW
Unit of Work	core/uow.py	Gestión de transacción: abre la sesión de BD, provee acceso a todos los repositorios, hace commit() automático al salir sin excepciones o rollback() si ocurre error.	Repository, Session
Repository	repository.py	Acceso a BD: queries sin lógica de negocio. Hereda de BaseRepository[T] genérico. Recibe la sesión del UoW por inyección.	Model, Session
Model	model.py	SQLModel tables + relaciones. Sin imports de capas superiores. Define la estructura de la base de datos.	Ninguna

2.2 Capas del Frontend — Feature-Sliced Design

El frontend aplica Feature-Sliced Design. Cada feature es autocontenida: sus componentes, hooks y estilos no son accesibles desde otras features. Los imports fluyen de arriba hacia abajo: Pages → Features → Hooks/Stores → API → Types.

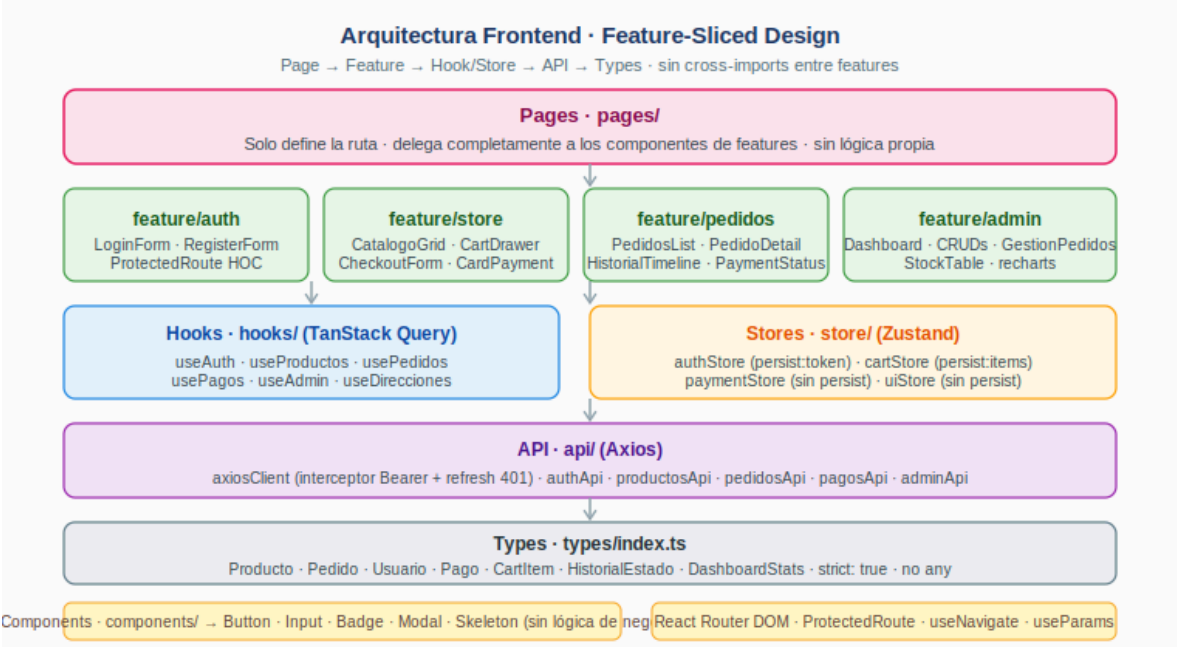


Figura 2 — Capas del frontend y organización feature-sliced

Separación Zustand / TanStack Query:
Zustand gestiona el estado del CLIENTE: carrito, sesión, proceso de pago, UI local (modales, sidebar).
TanStack Query gestiona el estado del SERVIDOR: productos, pedidos, dashboard. Datos remotos con caché automático.
Mezclar ambos tipos de estado en el mismo store es un error arquitectónico que debe evitarse.

2.3 Módulos Backend (Feature-First)

Módulo	Ruta	Descripción
Auth	app/modules/auth/	Login, registro, refresh, logout. JWT access (30 min) + refresh (7 días). Rate limiting.
Refreshtokens	app/modules/refreshtokens/	Modelo RefreshToken en BD para invalidación segura en logout.
Usuarios	app/modules/usuarios/	CRUD usuarios + asignación de roles RBAC. Soft delete.
Direcciones	app/modules/direcciones/	CRUD completo DireccionEntrega por usuario. PATCH /principal.
Categorías	app/modules/categorias/	Categorías jerárquicas con CTE recursiva. Soft delete con validación.
Productos	app/modules/productos/	Catálogo con Ingrediente (es_alergeno). Stock como campo en Producto.
Pedidos	app/modules/pedidos/	Dominio central: máquina de estados FSM, audit trail, historial append-only.

Módulo	Ruta	Descripción
Pagos	app/modules/pagos/	Integración MercadoPago: crear pago, webhook IPN, registro de transacciones.
admin	app/modules/admin/	Dashboard con métricas, gestión de stock y usuarios desde el panel.

3. Modelo de Datos — ERD v5

El esquema aplica Tercera Forma Normal (3FN), Soft Delete (deleted_at TIMESTAMPTZ), Snapshot Pattern en pedidos y Audit Trail append-only en HistorialEstadoPedido. La versión 5 incorpora todas las correcciones de auditoría: RefreshToken, Ingrediente, ProductoIngrediente, Pago completo y correcciones de tipos.

3.1 Dominio 1 — Identidad y Acceso

Entidad	Campo clave	Tipo	Restricción	Notas
Usuario	id	BIGSERIAL	PK	Soft-delete vía deleted_at
Usuario	email	VARCHAR(254)	UQ, NN	Validar con EmailStr (Pydantic v2)
Usuario	password_hash	CHAR(60)	NN	bcrypt cost≥12. NUNCA almacenar plaintext
Rol	codigo	VARCHAR(20)	PK (semántica)	ADMIN STOCK PEDIDOS CLIENT
UsuarioRol	(usuario_id, rol_codigo)	BIGINT + VARCHAR	PK compuesta	Pivot N:M. Incluye asignado_p or_id
RefreshToken ★	token_hash	CHAR(64)	UQ, NN	SHA-256 del token. revoked_at NULL = activo
RefreshToken ★	expires_at	TIMESTAMPTZ	NN	7 días desde emisión
RefreshToken ★	revoked_at	TIMESTAMPTZ	NULL	Se completa en POST /auth/logout
DireccionEntrega ★	alias	VARCHAR(50)	NULL	Ej: 'Casa', 'Trabajo'
DireccionEntrega ★	linea1	TEXT	NN	
DireccionEntrega ★	es_principal	BOOLEAN	NN, default false	Solo una por usuario

3.2 Dominio 2 — Catálogo de Productos

Entidad	Campo clave	Tipo	Restricción	Notas
Categoria	parent_id	BIGINT	FK self-ref, NULL	Jerarquía recursiva. ON DELETE SET NULL. CTE.
Producto	precio_base	DECIMAL(10,2)	CHECK ≥ 0, NN	Snapshot al crear pedido

Entidad	Campo clave	Tipo	Restricción	Notas
Producto ★	stock_cantidad	INTEGER	CHECK ≥ 0 , NN, default 0	Gestionado por rol STOCK
Producto ★	disponible	BOOLEAN	NN, default true	Toggle manual independiente del stock
Ingrediente ★	nombre	VARCHAR(100)	UQ, NN	Especificación completa en v5
Ingrediente ★	es_alergeno	BOOLEAN	NN, default false	Badge de alérgenos en UI
ProductoCategoria	(producto_id, cat_id)	BIGINT×2	PK compuesta	Pivot N:M. es_principal.
ProductoIngrediente ★	es_removible	BOOLEAN	NN	Habilita personalización del pedido
FormaPago ★	codigo	VARCHAR(20)	PK semántica	MERCADO PAGO EFECTIVO TRANSFERENCIA
FormaPago ★	habilitado	BOOLEAN	NN, default true	Se puede deshabilitar sin eliminar

3.3 Dominio 3 — Ventas, Pagos y Trazabilidad

Entidad	Campo clave	Tipo	Restricción	Notas
EstadoPedido	codigo	VARCHAR(20)	PK semántica	Catálogo. Ver máquina de estados.
EstadoPedido	es_terminal	BOOLEAN	NN	true = no admite transiciones salientes
Pedido	estado_codigo	VARCHAR(20)	FK → EstadoPedido	Estado actual del pedido
Pedido	total	DECIMAL(10,2)	CHECK ≥ 0 , NN	Snapshot inmutable al crear
Pedido	costo_envio	DECIMAL(10,2)	NN, default 50.00	Valor fijo v1. Documentado.
Pedido	forma_pago_codigo	VARCHAR(20)	FK → FormaPago	Alineado con PK semántica
Pedido	direccion_id	BIGINT	FK, SET NULL	NULL = retiro en local (válido)
DetallePedido	nombre_snapshot	VARCHAR(200)	NN, snap	Snapshot: nombre al

Entidad	Campo clave	Tipo	Restricción	Notas
				crear. Immutable.
DetallePedido	precio_snapshot	DECIMAL(10,2)	NN, snap	Snapshot: precio al crear. Immutable.
DetallePedido	personalizacion	INTEGER[]	NULL	IDs de ingredientes removidos (v5: INTEGER[])
HistorialEstadoPedido	estado_desde	VARCHAR(20)	FK, NULL	NULL = transición inicial (RN-02)
HistorialEstadoPedido	created_at	TIMESTAMPTZ	NN, append-only	Nunca updated_at. Append-only (RN-03).
Pago ★	mp_payment_id	BIGINT	UQ, NULL	ID devuelto por MercadoPa go
Pago ★	mp_status	VARCHAR(30)	NN	pending / approved / rejected
Pago ★	external_reference	VARCHAR(100)	UQ, NN	UUID del Pedido como referencia MP
Pago ★	idempotency_key	VARCHAR(100)	UQ, NN	UUID generado por backend. Evita cobros duplicados.

3.4 Máquina de Estados — Pedido

La entidad EstadoPedido es un catálogo. La capa de servicio valida la transición contra el mapa definido antes de cada INSERT en HistorialEstadoPedido. Ningún router puede saltar esta validación.



Figura 3 — Máquina de estados del Pedido (FSM). Las flechas rojas punteadas indican transición → CANCELADO

Código	Descripción	Orden	es_terminal	Transiciones válidas
PENDIENTE	Pedido creado, pago pendiente	1	false	→ CONFIRMADO, → CANCELADO
CONFIRMADO	Pago procesado y confirmado	2	false	→ EN_PREP, → CANCELADO
EN_PREP	En preparación en cocina	3	false	→ EN_CAMINO, → CANCELADO (solo ADMIN/PEDIDOS)
EN_CAMINO	Despachado al cliente	4	false	→ ENTREGADO
ENTREGADO	Entrega confirmada	5	TRUE ✓	— (estado terminal)
CANCELADO	Pedido cancelado	6	TRUE ✓	— (estado terminal)

Reglas de negocio — Pedidos:

RN-01: Un estado con es_terminal = true no admite transiciones salientes. Validación en Service.

RN-02: El primer registro de HistorialEstadoPedido siempre tiene estado_desde = NULL.

RN-03: La tabla HistorialEstadoPedido es append-only. Ninguna capa puede emitir UPDATE ni DELETE.

RN-04: El total, nombre y precio en DetallePedido son un snapshot inmutable al crear el pedido.

RN-05: El motivo es obligatorio si nuevo_estado = CANCELADO.

4. Autenticación y Autorización

4.1 Flujo de Autenticación

Pas o	Actor	Acción	Resultado esperado
1	Cliente	POST /api/v1/auth/login con email + password	HTTP 200 + access token (30 min) + refresh token (7 días)
2	Frontend	Almacena access token en authStore (Zustand). NO en localStorage directamente.	Token disponible para interceptor Axios
3	Frontend	Interceptor Axios agrega Authorization: Bearer <token> a cada request	Request autenticado hacia el backend
4	Backend	Dependency get_current_user() valida JWT y carga el usuario	Objeto usuario inyectado en el handler
5	Backend	require_role([Rol.ADMIN]) verifica roles del token	HTTP 403 si rol insuficiente
6	Cliente	POST /api/v1/auth/refresh con refresh token	Nuevo access token sin requerir re-login
7	Cliente	POST /api/v1/auth/logout	Refresh token marcado como revoked_at en tabla RefreshToken

4.2 Roles y Permisos (RBAC)

Rol	Código	Permisos principales	Restricciones
Administrador	ADMIN	CRUD completo: usuarios, categorías, productos, pedidos, stock. Asigna roles.	Sin restricciones.
Gestor de Stock	STOCK	Leer productos, actualizar stock_cantidad y disponible, ver ingredientes.	Sin acceso a usuarios ni datos financieros.
Gestor de Pedidos	PEDIDOS	Ver todos los pedidos, avanzar estados CONFIRMADO → ENTREGADO, ver historial.	Sin acceso a productos ni finanzas.
Cliente	CLIENT	Ver catálogo, gestionar carrito, crear pedidos, ver sus propios pedidos.	Solo accede a sus propios datos.

4.3 Rate Limiting en Autenticación

Configuración	Valor
Librería	slowapi — integración nativa con FastAPI
Límite	5 intentos fallidos por dirección IP en 15 minutos
Endpoint protegido	POST /api/v1/auth/login
Respuesta al superar	HTTP 429 Too Many Requests con header Retry-After
Configuración	Middleware global en app/main.py + decorador @limiter.limit() en el router de auth

5. Especificación de API REST

Todos los endpoints usan el prefijo `/api/v1`. Los errores siguen RFC 7807 (Problem Details). La documentación interactiva se genera automáticamente en `/docs` (Swagger UI) y `/redoc`.

Convenciones globales:

Error estándar RFC 7807: { "detail": "mensaje", "code": "ERROR_CODE", "field": "campo_opcional" }

Paginación: GET `/recursos?page=1&size=20` → { "items": [...], "total": N, "page": 1, "size": 20, "pages": P }

Soft delete: todos los GET filtran WHERE `deleted_at IS NULL`. Los registros eliminados no son visibles.

5.1 Módulo Auth

Método	Endpoint	Body / Params	Response	Auth requerida
POST	<code>/api/v1/auth/register</code>	{ nombre, apellido, email, password }	201 UserResponse	No
POST	<code>/api/v1/auth/login</code>	{ email, password }	200 TokenResponse	No — rate limited 5/15min
POST	<code>/api/v1/auth/refresh</code>	{ refresh_token }	200 TokenResponse	No
POST	<code>/api/v1/auth/logout</code>	{ refresh_token }	204 No Content	Bearer token
GET	<code>/api/v1/auth/me</code>	—	200 UserResponse	Bearer token

5.2 Módulo Productos

Método	Endpoint	Descripción	Rol requerido	Response
GET	<code>/api/v1/productos</code>	Listar (filtro: categoria, disponible, search, page, size)	Público	200 PaginatedProductos
GET	<code>/api/v1/productos/{id}</code>	Detalle con ingredientes, categorías y stock	Público	200 ProductoDetail
POST	<code>/api/v1/productos</code>	Crear producto con ingredientes y categorías	ADMIN	201 ProductoRead
PUT	<code>/api/v1/productos/{id}</code>	Actualizar producto	ADMIN	200 ProductoRead
PATCH	<code>/api/v1/productos/{id}/disponibilidad</code>	Cambiar disponible (true/false)	ADMIN, STOCK	200 ProductoRead
DELETE	<code>/api/v1/productos/{id}</code>	Soft delete producto	ADMIN	204 No Content
GET	<code>/api/v1/productos/{id}/ingredientes</code>	Listar ingredientes del producto	Público	200 List[IngredientRead]
POST	<code>/api/v1/productos/{id}/ingredientes</code>	Asociar ingrediente a producto	ADMIN	201 ProductoIngredienteRead
DELETE	<code>/api/v1/productos/{id}/ingredientes/{ing_id}</code>	Quitar ingrediente de producto	ADMIN	204 No Content

5.3 Módulo Pedidos

Método	Endpoint	Descripción	Rol requerido	Response
GET	/api/v1/pedidos	Listar propios (CLIENT) o todos (ADMIN/PEDIDOS)	CLIENT/ADMIN/PEDIDOS	200 Paginated Pedidos
GET	/api/v1/pedidos/{id}	Detalle completo con líneas, trazabilidad y estado de pago	Propietario/ADMIN	200 PedidoDetail
POST	/api/v1/pedidos	Crear pedido desde carrito. Todo en una transacción (UoW).	CLIENT	201 PedidoRead
PATCH	/api/v1/pedidos/{id}/estado	Avanzar estado. Valida FSM. UoW atómico.	ADMIN/PEDIDOS	200 PedidoRead
GET	/api/v1/pedidos/{id}/historial	Historial completo. ORDER BY created_at ASC.	Propietario/ADMIN	200 List[HistorialRead]
DELETE	/api/v1/pedidos/{id}	Cancelar propio (solo PENDIENTE o CONFIRMADO).	CLIENT propietario	200 PedidoRead

5.4 Módulo Pagos (MercadoPago) ★

Método	Endpoint	Descripción	Rol requerido	Response
POST	/api/v1/pagos/crear	Crea pago con token de tarjeta. Registra en tabla Pago.	CLIENT	201 PagoResponse
POST	/api/v1/pagos/webhook	Endpoint IPN de MercadoPago. Actualiza estado del pago y del pedido.	Público (validar firma)	200 { status: ok }
GET	/api/v1/pagos/{pedido_id}	Consulta el pago asociado a un pedido.	Propietario/ADMIN	200 PagoResponse

6. Schemas de Request / Response (Pydantic v2)

Todos los schemas usan Pydantic v2. Se definen schemas separados para Create, Update y Read. Nunca se expone el model de SQLAlchemy directamente como response.

6.1 Auth

Schema	Campos requeridos	Validaciones
LoginRequest	email: EmailStr, password: str	password mínimo 8 caracteres
RegisterRequest	nombre, apellido, email: EmailStr, password: str	nombre/apellido min 2 max 80. password min 8. Unicidad de email en servicio.
TokenResponse	access_token, refresh_token, token_type, expires_in: int	token_type = 'bearer'. expires_in en segundos.
UserResponse	id, nombre, apellido, email, roles: list[str], created_at	Nunca incluye password_hash.

6.2 Pedidos

Schema	Campos	Validaciones / Notas
CrearPedidoRequest	items: list[ItemPedidoRequest], forma_pago_codigo: str, direccion_id: int None, notas: str None	Mínimo 1 item. forma_pago_codigo debe existir en catálogo.
ItemPedidoRequest	producto_id: int, cantidad: int, personalizacion: list[int] None	cantidad ≥ 1. personalizacion = IDs de ingredientes removidos (INTEGER[]).
AvanzarEstadoRequest	nuevo_estado: str, motivo: str None	motivo obligatorio si nuevo_estado = CANCELADO (RN-05).
PedidoRead	id, estado_codigo, total, created_at	Versión compacta para listados.
PedidoDetail	id, estado_codigo, subtotal, descuento, costo_envio, total, items, historial, pago	Versión completa para vista de detalle.
DetallePedidoRead	producto_id, nombre_snapshot, precio_snapshot, cantidad, personalización	Snapshot: precio y nombre no reflejan cambios posteriores.

7. Patrón Unit of Work (UoW)

El Unit of Work actúa como director de orquesta que garantiza que todas las operaciones de base de datos dentro de una transacción de negocio tengan éxito o fallen como un conjunto. El commit ya no ocurre en el service.

7.1 Flujo de una Operación con UoW — Crear Pedido

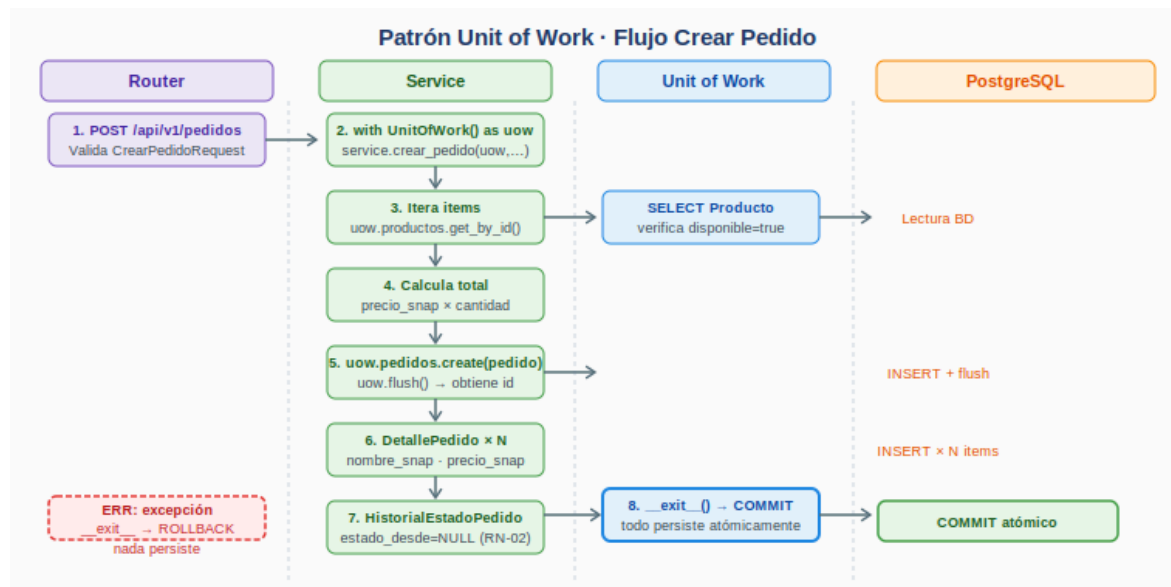


Figura 4 — Flujo de creación de pedido con Unit of Work. Todos los INSERT son atómicos.

Pas o	Capa	Operación	¿Toca BD?
1	Router	Recibe POST /api/v1/pedidos. Valida body con CrearPedidoRequest.	No
2	Router	Abre contexto: with UnitOfWork() as uow: — llama service.crear_pedido(uow, body, usuario_id).	No
3	Service	Itera items. Para cada uno: uow.productos.get_by_id(). Verifica disponible = true.	Lectura
4	Service	Calcula total como suma de precio_snapshot × cantidad.	No
5	Service	Llama uow.pedidos.create(pedido). uow.flush() → obtiene pedido.id.	INSERT + flush
6	Service	Crea DetallePedido por cada item con nombre_snapshot y precio_snapshot.	INSERT × N
7	Service	Crea primer HistorialEstadoPedido con estado_desde=None (RN-02).	INSERT
8	UoW	__exit__ sin excepción → session.commit(). Todo persiste atómicamente.	COMMIT
9	Router	Serializa pedido con PedidoRead.model_validate(pedido). Retorna HTTP 201.	No
ERR	UoW	Si cualquier paso 3-7 lanza excepción → __exit__ llama rollback(). Nada persiste.	ROLLBACK

7.2 BaseRepository[T] Genérico

Método	Descripción
<code>get_by_id(entity_id: int) → T None</code>	Obtiene entidad por clave primaria. Retorna None si no existe.
<code>list_all(skip: int, limit: int) → list[T]</code>	Listado simple sin filtros. Para listados complejos, el repo específico define su propio método.
<code>count() → int</code>	Cantidad total de registros. Útil para paginación.
<code>create(entity: T) → T</code>	Agrega a sesión + <code>flush()</code> + <code>refresh()</code> . Retorna entidad con ID asignado.
<code>update(entity: T) → T</code>	Agrega entidad modificada a sesión + <code>flush()</code> + <code>refresh()</code> .
<code>soft_delete(entity: T) → None</code>	Asigna <code>deleted_at = now()</code> . Solo para entidades con <code>soft-delete</code> .
<code>hard_delete(entity: T) → None</code>	Hard delete. Solo se usa cuando el modelo no tiene <code>soft-delete</code> .

8. Integración MercadoPago

Food Store integra MercadoPago Checkout API. Permite procesar pagos con tarjeta de crédito/débito, Rapipago, Pago Fácil y Cuenta MercadoPago sin redirigir al cliente fuera del sitio.

¿Por qué Checkout API con Orders?

Única integración para múltiples medios de pago. Sin múltiples APIs separadas.

Datos de tarjeta tokenizados por MercadoPago.js — NUNCA pasan por el servidor de Food Store (PCI SAQ-A).

Notificaciones push (IPN/webhook) para confirmación asíncrona del pago.

idempotency_key UUID generado por el backend evita cobros duplicados por reintento.

8.1 Flujo Completo de Pago

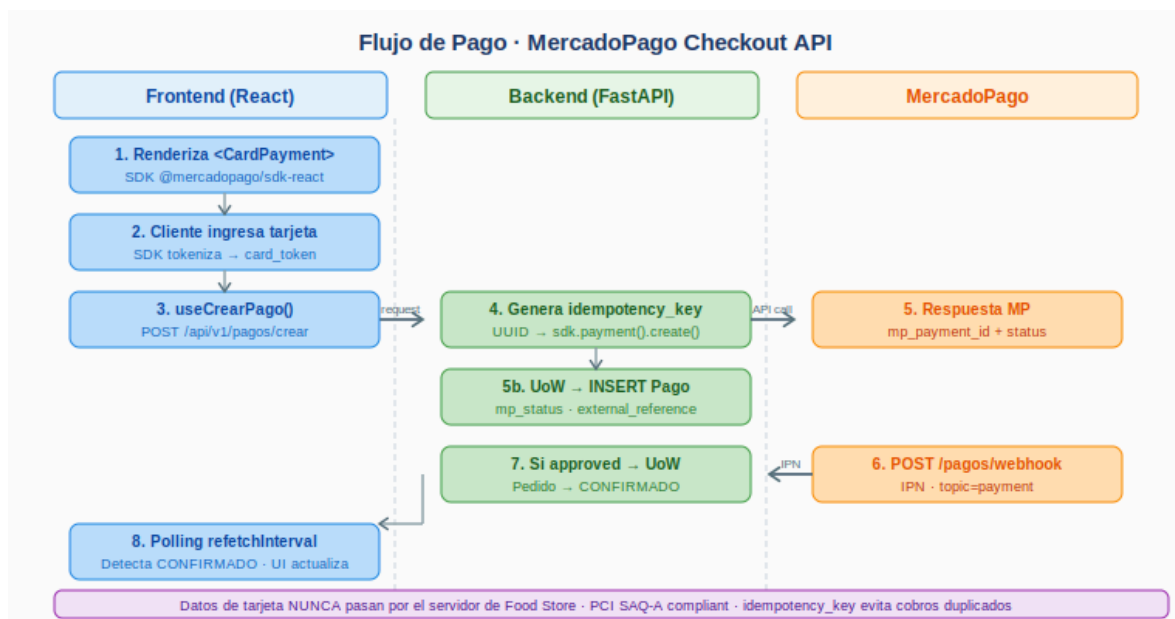


Figura 5 — Flujo completo de pago MercadoPago. Los datos de tarjeta nunca pasan por Food Store.

8.2 Estados de Pago y Acciones del Sistema

Estado MP	Descripción	Acción en Food Store
approved	Pago aprobado y acreditado	Webhook avanza el pedido a CONFIRMADO de forma automática vía UoW.
pending	Pago iniciado o en proceso (efectivo pendiente)	Pedido permanece en PENDIENTE. El webhook confirmará cuando se acredite.
rejected	Pago rechazado por el banco o MP	Se muestra status_detail al cliente. El pedido permanece en PENDIENTE.
in_process	Pago en revisión manual por MP	Pedido permanece en PENDIENTE. El webhook notificará la resolución.
cancelled	Pago cancelado	El cliente puede reintentar o cancelar el pedido.

8.3 Tarjetas de Prueba — Sandbox

Número de tarjeta	Red	Resultado	CVV	Vencimiento
4509 9535 6623 3704	Visa	Pago aprobado	123	11/25

Número de tarjeta	Red	Resultado	CVV	Vencimiento
3714 496353 98431	American Express	Pago aprobado	1234	11/25
4000 0000 0000 0002	Visa	Pago rechazado	123	11/25

9. Gestión de Estado con Zustand

Zustand es la librería de gestión de estado global del frontend. Food Store requiere cuatro stores con responsabilidades claramente separadas.

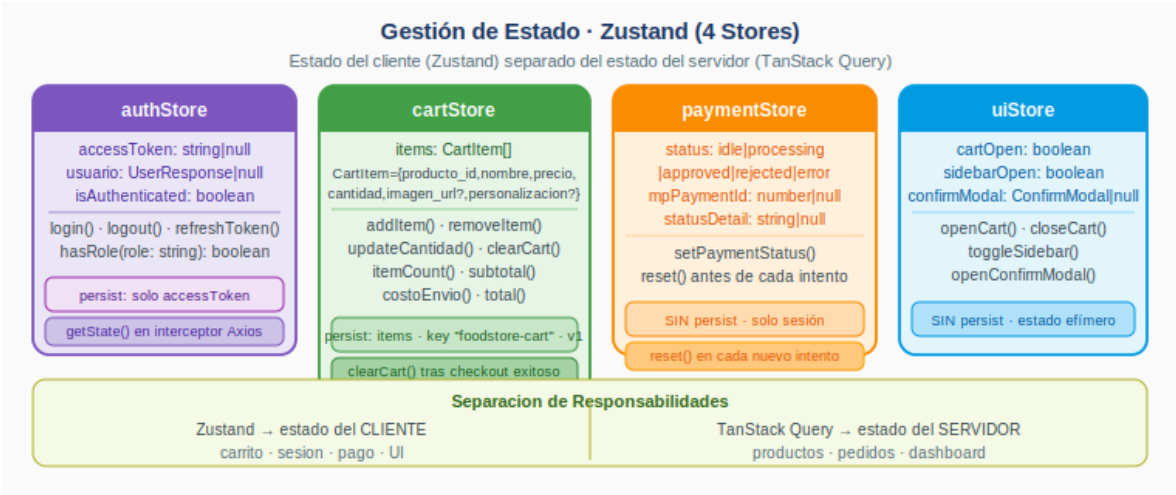


Figura 6 — Los cuatro stores Zustand y sus responsabilidades. Persistencia selectiva por store.

Store	Archivo	Estado que gestiona	Middleware	Persiste
authStore	store/authStore.ts	accessToken, usuario, isAuthenticated	persist	Sí — solo el accessToken
cartStore	store/cartStore.ts	items del carrito, cantidades, personalizaciones	persist	Sí — items completos (v1)
paymentStore	store/paymentStore.ts	Estado del proceso de pago MP: status, mpPaymentId	Ninguno (sesión)	No — se resetea al recargar
uiStore	store/uiStore.ts	cartOpen, sidebarOpen, confirmModal activo	Ninguno	No

Buenas prácticas de consumo de stores:

Suscripción por slice: `const itemCount = useCartStore(s => s.itemCount())` — evita re-renders innecesarios.

Actions extraídas sin re-render: `const { addItem } = useCartStore()`

Nunca suscribirse al store completo sin selector: ❌ `const store = useCartStore()`

Acceso fuera de React (interceptores): `useAuthStore.getState().accessToken`

authStore: `partialize` → solo `accessToken`. Al recargar, `GET /api/v1/auth/me` reconstruye usuario.

10. Configuración y Setup

10.1 Variables de Entorno

Variable	Descripción	Valor ejemplo
DATABASE_URL	Conexión a PostgreSQL	postgresql://user:pass@localhost:5432/foodstore_db
SECRET_KEY	Clave secreta para firmar JWT (mín. 32 chars)	your-super-secret-key-min-32-char s
ALGORITHM	Algoritmo JWT	HS256
ACCESS_TOKEN_EXPIRE_MINUTES	Expiración del access token en minutos	30
REFRESH_TOKEN_EXPIRE_DAYS	Expiración del refresh token en días	7
CORS_ORIGINS	Orígenes permitidos (JSON array)	["http://localhost:5173"]
MP_ACCESS_TOKEN	Access Token de MercadoPago (backend)	TEST-xxxx
MP_PUBLIC_KEY	Public Key de MercadoPago (para el frontend)	TEST-xxxx
MP_NOTIFICATION_URL	URL del webhook IPN de MercadoPago	https://dominio.com/api/v1/pagos/webhook
VITE_API_URL	URL base del backend (Vite — frontend)	http://localhost:8000
VITE_MP_PUBLIC_KEY	Public Key MP expuesta al frontend vía Vite	TEST-xxxx

10.2 Seed Data Obligatorio — app/db/seed.py

El script seed.py debe ejecutarse una vez después de alembic upgrade head. Sin este paso, la aplicación no funciona: no existen roles, estados de pedido ni formas de pago.

Entidad	Registros a insertar
Rol	ADMIN, STOCK, PEDIDOS, CLIENT — los cuatro roles del sistema RBAC
EstadoPedido	PENDIENTE, CONFIRMADO, EN_PREP, EN_CAMINO, ENTREGADO, CANCELADO — con es_terminal correspondiente
FormaPago	MERCADOPAGO (habilitado), EFECTIVO (habilitado), TRANSFERENCIA (habilitado)
Usuario admin	admin@foodstore.com / Admin1234! — con rol ADMIN asignado. Contraseña debe cambiarse en producción.

11. Patrones Aplicados en el Proyecto

Patrón	Capa	Descripción
Repository Pattern	Backend	Abstracción del acceso a BD. BaseRepository[T] genérico. Facilita testing con mocks.
Unit of Work	Backend	Gestión de transacciones atómicas. El Service opera dentro del contexto UoW sin gestionar la sesión directamente.
Service Layer	Backend	Lógica de negocio centralizada, stateless. Consume el UoW. Independiente del framework.
Snapshot Pattern	Backend/BD	Precios y nombres de producto inmutables al crear el pedido. Garantiza integridad histórica.
Soft Delete	Backend/BD	deleted_at TIMESTAMPTZ — registros lógicamente eliminados. Nunca DELETE físico en entidades de negocio.
Audit Trail Append-Only	Backend/BD	HistorialEstadoPedido: solo INSERT, nunca UPDATE/DELETE (RN-03). Trazabilidad completa.
State Machine (FSM)	Backend	Transiciones del pedido validadas en la capa de servicio contra el mapa de transiciones permitidas.
Idempotent Payments	Backend	UUID como idempotency_key enviado a MercadoPago. Evita cobros duplicados por reintentos.
Feature-Sliced Design	Frontend	Organización por features con límites de importación claros. Cada feature es autocontenida.
Custom Hooks	Frontend	Encapsulan lógica de TanStack Query en hooks reutilizables por dominio.
Optimistic Updates	Frontend	Actualización inmediata de UI antes de confirmar respuesta del servidor. Rollback en error.
Webhook / IPN	Backend	MercadoPago notifica de forma asíncrona el resultado del pago. Evita polling constante.

12. Rúbrica de Corrección — v5.0

Puntaje total: 200 puntos. Corrección escrita + video de demostración obligatorio.

Criterio	Pts	Excelente	Bueno	Regular	Insuficiente
Backend — Estructura y Configuración	10	Capas router/service/uow/repository/model. Módulos por dominio. core/ separado. Alembic + seed. CORS + rate limiting.	Separación parcial. Seed incompleto.	Estructura plana o sin capas.	Sin estructura reconocible.
Backend — Modelo de Datos	15	SQLModel correcto, constraints, soft-delete, snapshot, entidades completas (Ingrediente, FormaPago, DireccionEntrega, RefreshToken, Pago).	Correctos, faltan algunas entidades.	Básicos sin patrones avanzados.	Incorrectos o incompletos.
Backend — Unit of Work y Repository	15	UoW completo con context manager, commit/rollback automático. BaseRepository[T] genérico. Ningún service hace session.commit().	UoW presente pero incompleto.	Sin UoW, transacciones manuales.	Sin gestión de transacciones.
Backend — Capa de Servicio	15	FSM implementada. RN-01/02/03/05 validadas. Services stateless. Service recibe uow por parámetro.	Lógica presente pero algunas RN en routers.	Lógica básica sin validaciones.	Sin capa de servicio.
Backend — Controladores REST	15	Verbos HTTP correctos, rutas semánticas, status codes precisos (201/204/422/402), prefijo /api/v1. Schemas Pydantic separados Read/Create/Update.	Verbos y rutas correctas, algunos status codes incorrectos.	Verbos incorrectos o rutas no semánticas.	Sin convenciones REST.
Backend — MercadoPago	15	SDK Python configurado con idempotency_key UUID. Webhook /pagos/webhook que procesa topic=payment y avanza pedido. Tabla Pago completa.	SDK configurado, sin idempotency_key.	Integración parcial o hardcodeada.	Sin integración MP.
Frontend — Estructura y TypeScript	10	Feature-sliced: pages/features/components/hooks/store/api/types. Sin cross-imports. strict: true, no any.	Estructura presente, algunos módulos mezclados.	Estructura plana.	Sin estructura.
Frontend — Zustand	10	4 stores implementados y tipados. persist correcto por store. Suscripción por slice.	3 stores correctos.	Solo authStore y cartStore básicos.	Sin Zustand.

Criterio	Pts	Excelente	Bueno	Regular	Insuficiente
Frontend — TanStack Query	15	useQuery/useMutation para todo el fetch. queryKeys descriptivos. Invalidación tras mutaciones. Interceptor refresh 401 automático.	TanStack Query presente, invalidación parcial.	Fetch directo con useEffect.	Sin TanStack Query.
Frontend — Funcionalidades Cliente	15	Catálogo con debounce/filtros/paginación/skeleton. Carrito persist. Checkout con CardPayment de MP. Timeline con polling 30s.	Funcional con algunas carencias.	Lista sin filtros ni paginación.	Sin funcionalidades.
Frontend — Panel Admin	15	Dashboard KPIs + recharts. CRUD categorías/productos con relaciones. Gestión pedidos con FSM. Gestión stock.	CRUD funcional, falta alguna feature.	Solo visualización.	Sin panel admin.
UI/UX y Diseño	10	Sistema de diseño consistente. Mobile-first. Skeleton loaders, toasts, modales de confirmación, estados vacíos.	Diseño consistente con pequeñas inconsistencias.	Diseño básico sin sistema.	Sin diseño coherente.
Calidad de Código	10	snake_case/camelCase/PascalCase. Funciones < 50 líneas. SRP. Docstrings. JSDoc. README.md completo.	Nomenclatura correcta, algunas funciones largas.	Mezcla de convenciones.	Sin convenciones.

Escala de calificación:

181-200 pts (90-100%) — EXCELENTE: Proyecto completo, profesional, con todas las capas y buenas prácticas.

141-180 pts (70-89%) — BUENO: Proyecto funcional con pequeños ajustes o funcionalidades faltantes.

101-140 pts (50-69%) — REGULAR: Proyecto básico con errores o funcionalidades incompletas.

0-100 pts (0-49%) — INSUFICIENTE: Proyecto incompleto, no funcional o no sigue la especificación.

Bonus +10 pts: Tests unitarios con pytest, cobertura > 60% (test_pedidos, test_pagos, test_auth).

Bonus +10 pts: Deploy funcional en Railway, Render o Fly.io con URL accesible.

⚠ Penalización -30%: El proyecto que no corra localmente siguiendo el README.

13. Entrega del Proyecto

13.1 Checklist de Entrega

Ítem	Descripción	Estado
CE-01	Link a repositorio GitHub público en la entrega	☐ Pendiente
CE-02	README.md con instrucciones de setup funcionando en máquina limpia	☐ Pendiente
CE-03	.env.example completo con variables de MercadoPago documentadas	☐ Pendiente
CE-04	alembic upgrade head sin errores	☐ Pendiente
CE-05	python -m app.db.seed ejecuta correctamente y carga datos iniciales	☐ Pendiente
CE-06	npm install + npm run dev sin errores	☐ Pendiente
CE-07	pip install -r requirements.txt + uvicorn app.main:app sin errores	☐ Pendiente
CE-08	Swagger UI (/docs) accesible con todos los endpoints documentados	☐ Pendiente
CE-09	Pago de prueba con tarjeta sandbox MP funciona end-to-end	☐ Pendiente
CE-10	Unit of Work correctamente implementado (ningún service.session.commit() directo)	☐ Pendiente
CE-11	4 Zustand stores implementados, tipados y con persist correcto	☐ Pendiente
CE-12	Screenshots de al menos 10 pantallas distintas	☐ Pendiente
CE-13	Link a video demostración (5-10 min) en README	☐ Pendiente
CE-14	Repositorio público verificado con sesión cerrada	☐ Pendiente

Apéndice — Referencias y Recursos

Tecnología	URL
FastAPI	https://fastapi.tiangolo.com
SQLModel	https://sqlmodel.tiangolo.com
Pydantic v2	https://docs.pydantic.dev
Alembic	https://alembic.sqlalchemy.org
TanStack Query v5	https://tanstack.com/query
TanStack Form	https://tanstack.com/form
Zustand	https://zustand-demo.pmnd.rs
Tailwind CSS	https://tailwindcss.com/docs
recharts	https://recharts.org
slowapi — rate limiting FastAPI	https://github.com/laurentS/slowapi
MercadoPago Developers (AR)	https://www.mercadopago.com.ar/developers/es
MercadoPago SDK Python	https://github.com/mercadopago/sdk-python
MercadoPago SDK React	https://github.com/mercadopago/sdk-react